SR University

AI Assisted Coding

Assignment 13.2 (Tuesday) **2403A51L40**

D: 3/10/2026 **Akhila Gundeti**

Lab 13: Code Refactoring – Improving Legacy Code with AI

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

Sample Code:

length = 8

width = 4

print("Area:", length * width)

print("Perimeter:", 2 * (length + width))

length = 12

width = 6

print("Area:", length * width)

print("Perimeter:", 2 * (length + width))

Prompt:

Analyze the following Python code and refactor it by removing repeated calculations.

Create a reusable function to compute area and perimeter.

Ensure the output remains the same and add proper docstrings.

Output:

```python
def calculate_area_perimeter(length, width):
    """
    Calculate the area and perimeter of a rectangle.

    Parameters:
    length (float): The length of the rectangle.
    width (float): The width of the rectangle.

    Returns:
    tuple: A tuple containing the area and perimeter of the rectangle.
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter
# First rectangle
length = 8
width = 4
area, perimeter = calculate_area_perimeter(length, width)
print("Area:", area)
print("Perimeter:", perimeter)
# Second rectangle
length = 12
width = 6
area, perimeter = calculate_area_perimeter(length, width)
print("Area:", area)
print("Perimeter:", perimeter)
```

SR University

```
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
```

Explanation: The refactored code uses descriptive variable names that clearly indicate their purpose, improving readability. Additionally, comments are added to explain each step of the calculation, making it easier for others to understand the code.

Task Description -2 (Refactoring – Modularizing Inline Calculations)

Sample Code:

amount = 1000

discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)

amount = 2000

discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)

Prompt:

Refactor the following Python code by extracting repeated discount calculations into a reusable function.

Ensure readability and add a docstring.

Output:

```python
def calculate_final_amount(amount):
    """
    Calculate the final amount after applying a 10% discount.

    Parameters:
    amount (float): The original amount before discount.

    Returns:
    float: The final amount after applying the discount.
    """
    discount = amount * 0.10
    final_amount = amount - discount
    return final_amount
# First amount
amount = 1000
final_amount = calculate_final_amount(amount)
print("Final Amount:", final_amount)
# Second amount
amount = 2000
final_amount = calculate_final_amount(amount)
print("Final Amount:", final_amount)
```

SR University

```
Final Amount: 900.0
Final Amount: 1800.0
```

Explanation: The refactored code defines a function `calculate_area_perimeter` that computes both the area and perimeter of a rectangle, eliminating the need for repeated calculations. This improves readability and maintainability while keeping the output consistent.

Task Description -3 (Refactoring – Improving Loop and

Conditional Performance)

Sample Code:

students = ["Anil", "Bhavya", "Charan", "Divya"]

check_list = ["Charan", "Esha", "Bhavya"]

for name in check_list:

exists = False

for student in students:

if name == student:

exists = True

if exists:

print(name, "found")

else:

print(name, "not found")

Prompt:

Optimize the following Python code by removing inefficient nested loops.

Use a faster approach (such as sets) while keeping the same output

Output:

```python
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
student_set = set(students)
for name in check_list:
    if name in student_set:
        print(name, "found")
    else:
        print(name, "not found")
```

```
Charan found
Esha not found
Bhavya found
```

Explanation: The optimized code converts the list of students into a set, allowing for O(1) average time complexity for membership checks, thus eliminating the need for inefficient nested loops. This results in faster lookups while maintaining the same output.

Task Description -4 (Refactoring – Replacing Hardcoded Values

with Constants)

Sample Code:

print("Simple Interest:", (1000 * 2 * 5) / 100)

print("Simple Interest:", (2000 * 2 * 5) / 100)

Output:

```
PRINCIPAL_1 = 1000
PRINCIPAL_2 = 2000
RATE = 2
TIME = 5
simple_interest_1 = (PRINCIPAL_1 * RATE * TIME) / 100
simple_interest_2 = (PRINCIPAL_2 * RATE * TIME) / 100
print("Simple Interest:", simple_interest_1)
print("Simple Interest:", simple_interest_2)
```

```
Simple Interest: 100.0
Simple Interest: 200.0
```

Explanation: The refactored code defines named constants for the principal amounts, rate, and time, which enhances readability and makes it clear what each value represents. This approach also makes it easier to update the values in the future without having to search through the code for hardcoded numbers.

Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)

Sample Code:

x = 50

y = 30

z = x + y

print(z)

Output:

SR University

```python
# Define the first number
first_number = 50
# Define the second number
second_number = 30
# Calculate the sum of the two numbers
total = first_number + second_number
# Print the result
print(total)
```

```
80
```

Explanation: The refactored code uses descriptive variable names that clearly indicate their purpose, improving readability. Additionally, comments are added to explain each step of the calculation, making it easier for others to understand the code.